다이나믹 프로그래밍

권용환

2026 ALOHA 초급반 3주차

2026년 9월 28일

1. 재귀



- 다이나믹 프로그래밍을 알아보기 전에 재귀를 먼저 알아보자.
- 재귀란 자기 자신을 호출하는 행위이다.
- 오른쪽의 코드를 보면 함수 `f` 내에서 함수 `f` 를 다시 호출하고 있다.

```
void f() {
  // ...
  f();
  // ...
}
```



- 자기 자신을 호출하는 함수를 재귀함수라고 한다.
- 재귀함수를 구상할 때에는 호출 종료 조건을 설정하는 것이 중요하다.
- 호출한 함수가 자기 자신을 호출하고, 그 호출한 함수가 자기 자신을 호출하고..., 처럼 호출이 계속 반복되는 것을 막기 위해서이다.



- 재귀함수를 사용하는 가장 대표적인 예는 피보나치 수 구하기이다.
- 피보나치 수는 아래의 점화식으로 주어지는 수이다.

$$F_n = \begin{cases} F_{n-1} + F_{n-2} & (n \geq 2) \\ 1 & (n = 1) \\ 0 & (n = 0) \end{cases}$$

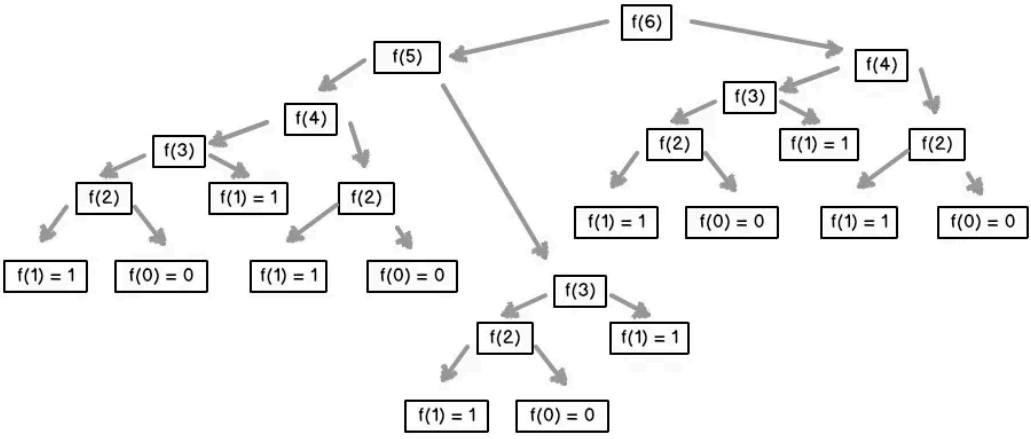
- 이때 n번째 피보나치 수를 구하는 함수 `fib(n)`를 구현하기 위하여 재귀를 사용할 수 있다.



```
int fib(int n) {
  if (n == 0) return 0;
  if (n == 1) return 1;
  return fib(n - 1) + fib(n - 2);
}
```

- 위 함수는 n번째 피보나치 수를 계산하는 함수이다. n이 1일 때 1, 0일 때 0을 반환한다.
- 나머지 경우에는 점화식에 따라 피보나치 수를 계산한다.
- 이처럼 전체 문제를 풀기 위한 부분 문제가 전체 문제와 같은 구조가 되는 경우가 있다.
- 이러한 경우에 전체 문제를 풀기 위하여 재귀를 사용할 수 있다.

1. 재귀


f(6)
f(5)
f(4)
f(4)
f(3)
f(3)
f(2)
f(1) = 1
f(2)
f(2)
f(1) = 1
f(0) = 0
f(1) = 1
f(0) = 0
f(2)
f(1) = 1
f(2)
f(1) = 1
f(0) = 0
f(1) = 1
f(0) = 0
f(3)
f(2)
f(1) = 1
f(1) = 1
f(0) = 0

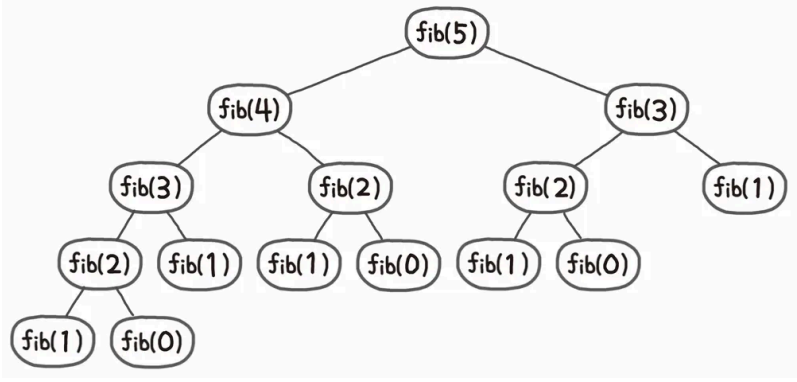


- 하지만 이렇게 재귀로 피보나치 수를 구한다면 n이 큰 수일 때 계산 속도가 상당히 느려진다.
- 앞의 그림을 보면 `f(6)`을 구할 때 `f(4)`는 2번, `f(3)`은 3번, `f(2)`는 5번이나 반복하여 호출되고 있다.
- 자세한 증명은 생략하지만, 이렇게 구현하면 시간 복잡도가 $\mathcal{O}(2^n)$이 되기 때문에 n이 1 증가할 때마다 계산 양이 대략 2배씩 많아지기 때문이다.

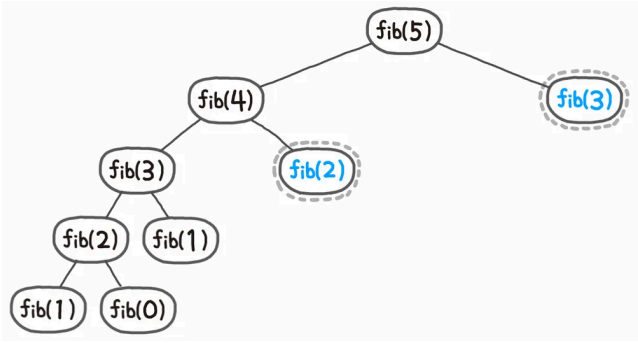


- 이때 반복되는 계산을 줄일 수 있는 방법이 있다.
- 예를 들어 `fib(2)` 값을 구했다면, 이 값을 저장해 놓고 다시 `fib(2)` 값이 필요할 때 계산하지 않고 꺼내 쓰는 방법이다.

- 재귀를 사용하여 `fib(5)`를 구한다면 아래와 같이 15번의 함수 호출을 하게 된다.



- ✓ 중복되는 부분(파란색으로 표시된 부분)을 기억하고 재활용한다면 아래와 같이 함수 호출 횟수를 9번으로 줄일 수 있다.





✓ 중복되는 부분을 재활용하는 피보나치 수 구하는 함수를 아래와 같이 구현하였다.

```
int a[11];
int fib(int n) {
  if (n == 1) return 1; // 초기값
  if (n == 0) return 0; // 초기값
  if (a[n] != -1) return a[n]; // 이미 구해진 값 재활용
  return a[n] = fib(n - 1) + fib(n - 2);
}
int main() {
  a[0] = 0; a[1] = 1;
  for (int i = 2; i <= 10; i++) a[i] = -1; // -1이면 아직 값이 구해지지 않음
  cout << fib(5) << "\n";
}
```



- 배열의 원소 `a[i]`는 `i`번째 피보나치 수의 값을 저장한다.
- 만약 `a[i]`가 `-1`이라면 아직 `i`번째 피보나치 수의 값을 구하지 않은 것이므로 점화식에 따라 구해 준다.
- `a[i]`가 `-1`이 아니라면 `i`번째 피보나치 수의 값은 이미 구해진 상태이므로 바로 `return`하여 추가적인 함수 호출을 막는다.



- 이와 같이 전체 문제를 구조가 동일한 여러 개의 부분 문제로 쪼개고, 각 부분 문제를 해결할 때 이미 계산한 값을 꺼내 와 계산 속도를 더 빠르게 하는 방법을 다이나믹 프로그래밍(DP)이라고 한다.
- 이 슬라이드에서 앞으로 다이나믹 프로그래밍은 DP라고 짧게 표시할 것이다.



- DP에서 상태(State)란 문제를 해결하기 위한 특정 시점의 상황이나 부분 문제의 결괏값을 의미한다.
- n번째 피보나치 수를 구하는 문제를 예로 들어 상태를 이해해 보자.
- 배열의 값 $\mathrm{dp}[i]$를 i번째 피보나치 수라고 정의해 보자.
- 5번째 피보나치 수를 구하기 위해서는 4번째 피보나치 수, 3번째 피보나치 수...가 필요하다.
- 따라서 $\mathrm{dp}[4]$, $\mathrm{dp}[3]$ 등은 부분 문제의 결괏값이므로 상태라고 할 수 있다.
- 상태를 정의할 때 쓰이는 변수의 개수에 따라 1차원, 2차원, 3차원 DP라고도 불린다.
- 예를 들어 $\mathrm{dp}[i][j]$와 같이 상태를 정의할 때 변수가 두 개 쓰이면 2차원 DP라고 부른다.



- DP로 문제를 풀 때에는 전체 문제를 부분 문제로 쪼개어서 푼다.
- 이때 이전 상태들(부분 문제)로 다음 상태(전체 문제)를 어떻게 만들 것인지를 생각해야 한다.
- 이를 상태 전이 혹은 점화식이라고 한다.
- 피보나치 수를 구할 때에는 이전 상태 $\mathrm{dp}[i-1]$과 $\mathrm{dp}[i-2]$를 활용하여 다음 상태 $\mathrm{dp}[i]$를 만든다.
- 이전 상태와 다음 상태의 관계는 $\mathrm{dp}[i] = \mathrm{dp}[i-1] + \mathrm{dp}[i-2]$이고, 이것이 점화식이다.



- ✓ 재귀함수를 만들 때 무한 반복을 피하기 위하여 초기값을 넣어야 했던 것이 기억나는가?
- ✓ DP에서도 초기값이 필요하다.
- ✓ 예를 들어 피보나치 수에서는 $\mathrm{dp}[0] = 0$이고 $\mathrm{dp}[1] = 1$이다.
- ✓ 초기값은 가장 작은 부분 문제의 정답이라고 볼 수도 있다.
- ✓ 이 초기값을 쌓아 올려 전체 문제의 정답을 만들게 된다.



- ✓ 상태를 정의하고, 점화식을 만들었고, 초기값을 정했다면 마지막으로 어떤 값이 우리가 구하는 정답인지 생각해야 한다.
- ✓ n번째 피보나치 수 계산에서는 $\mathrm{dp}[i]$를 i번째 피보나치 수로 정의하였으므로 우리가 구하는 정답은 $\mathrm{dp}[n]$이 된다.



- 이제 구현할 차례이다. DP를 구현할 때에는 두 가지 방법이 있다.
- 첫 번째는 Top-down 방식이다.
- 재귀를 활용하여 점화식의 값을 구한다.
- 위(큰 문제)에서부터 아래(작은 문제)로 점화식의 값을 구해 나가므로 Top-down이라는 이름이 붙었다.
- 11페이지의 코드가 피보나치 수 계산을 Top-down 방식으로 구현한 코드이다.



- ✓ 두 번째는 Bottom-up 방식이다.
- ✓ Top-down 방식과는 반대로 아래에서부터 위로 점화식의 값을 구한다.
- ✓ 재귀를 사용하지 않는다는 점이 특징이다.
- ✓ 아래 코드는 Bottom-up 방식으로 피보나치 수를 구하는 코드이다.

```
int a[11];
int main() {
  a[0] = 0; a[1] = 1; // 초기값
  for (int i = 2; i <= 10; i++) {
    a[i] = a[i - 1] + a[i - 2]; // 상태 전이
  }
  cout << a[5] << "\n"; // 결과 출력
}
```



- ✓ 지난 시간에 그리디 알고리즘을 활용하여 동전의 개수를 최소화하였다.
- ✓ 동전이 500원, 200원, 10원짜리였을 때 그리디 알고리즘이 동작하지 않은 것이 기억나는가?
- ✓ 이때 DP를 활용할 수 있다.
- ✓ 연습 문제 '동전교환'(JUNGOL 2000)을 풀어 보자.



- N개의 동전 금액과 만들어야 하는 잔돈 W가 주어진다.
- 잔돈 W를 최소의 동전 수로 만들어 보자.
- 저번 시간에도 알아보았듯이 그리디 알고리즘은 항상 동작하지 않는다.
- 그렇다면 모든 경우의 수를 다 확인해 보면 어떨까?



- ✓ 먼저 상태를 정의해 보자. $\mathrm{dp}[i]$를 i원을 만들기 위해 필요한 동전 개수의 최솟값으로 정의하자.
- ✓ 이렇게 하면 자연스럽게 $\mathrm{dp}[0] = 0$이라는 초기값도 정의된다.
- ✓ 정답인 $\mathrm{dp}[W]$에 접근할 수 있게 하기 위하여 dp 배열은 크기 $W + 1$ 이상으로 만들어야 한다.



- 이제 상태 전이를 생각해 보자. $\mathrm{dp}[a] = x$이고 동전의 종류는 100원짜리 하나밖에 없다고 생각해 보자.
- 그러면 $\mathrm{dp}[a + 100] = x + 1$일 것이다. 왜냐하면 a원을 만들기 위해 동전 x개가 필요했으니 $a + 100$원을 만들기 위해서는 100원 1개를 더하면 되기 때문이다.
- 초기값 $\mathrm{dp}[0] = 0$에서 $\mathrm{dp}[100] = 1$, $\mathrm{dp}[200] = 2$로 이어지는 관계를 생각하면 쉽게 이해할 수 있다.

- 만약 $\mathrm{dp}[a]$에서 $\mathrm{dp}[a+b]$로 전이하였는데 $\mathrm{dp}[a]+1$이 $\mathrm{dp}[a+b]$보다 크면 어떻게 될까?
- 즉, a원을 만드는 동전의 최소 개수에 b원짜리 동전 1개를 추가하여 $a+b$원을 만들었는데 이 개수가 이미 구해진 $a+b$원을 만드는 동전의 최소 개수보다 큰 것이다.
- 이러면 최솟값을 선택하는 것이 최적이다.



- 구체적인 예를 들어 보면 이해하기 더 쉽다.
- 100원짜리와 50원짜리 동전이 있다.
- 초기값 $\mathrm{dp}[0] = 0$에서 100원을 사용하여 $\mathrm{dp}[100] = 1$, $\mathrm{dp}[200] = 2$를 만들었다.
- 이후 50원을 사용하면 초기값에서 시작하여 $\mathrm{dp}[50] = 1$을 만들 수 있다.
- $\mathrm{dp}[50] = 1$에서 50원을 사용하면 $\mathrm{dp}[100] = 2$가 된다. 그런데 이미 $\mathrm{dp}[100]$ 값은 1로 구해져 있다.
- 이럴 때에는 구해진 두 값 중 최솟값을 고르면 된다.



- 각 동전의 값 c와 $0 \le i \le w$인 모든 i에 대하여 점화식은 아래와 같이 나타낼 수 있다.

$$\mathrm{dp}[i+c] = \min(\mathrm{dp}[i+c], \mathrm{dp}[i]+1) \quad (i+c \le W)$$

- 인덱스 범위를 벗어나지 않게 하기 위하여 $i+c \le W$ 조건이 있는 것에 유의하자.
- 또한 dp 배열에는 초기값을 제외하고 기본적으로 매우 큰 수를 저장한다.
- 잔돈 W를 만들 수 없는 경우를 확인하기 위해서이다.

2. 다이나믹 프로그래밍



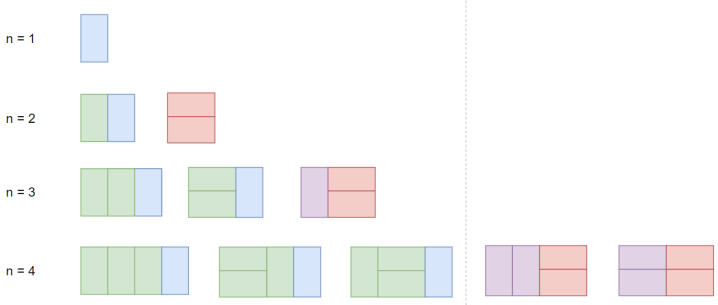
```cpp
#include <iostream>
using namespace std;
int dp[64005], c[10];
int main() {
    cin.tie(0)->sync_with_stdio(0);
    int n, w; cin >> n;
    for (int i = 0; i < n; i++) cin >> c[i];
    cin >> w;
    for (int i = 1; i <= w; i++) dp[i] = 1'000'000;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < w; j++) {
            if (j + c[i] > w) continue;
            dp[j + c[i]] = min(dp[j + c[i]], dp[j] + 1);
        }
    }
    if (dp[w] == 1'000'000) {
        cout << "impossible\n";
    } else {
        cout << dp[w] << "\n";
    }
    return 0;
}
```



✓ DP에는 여러 가지 자주 보이는 유형이 있다. 각 유형을 간단하게 알아보자.

$2 \times N$ 타일링

- ✓ 1×2 크기의 타일로 $2 \times N$ 크기의 공간을 덮는 경우의 수를 구하는 문제이다.





$2 \times N$ 타일링

- ✓ $\mathrm{dp}[n]$을 $2 \times n$ 크기의 공간을 덮는 경우의 수라고 정의하면 $\mathrm{dp}[n] = \mathrm{dp}[n-1] + \mathrm{dp}[n-2]$의 점화식을 얻을 수 있다. 우리가 구하는 정답은 $\mathrm{dp}[N]$이다.
- ✓ $2 \times n$ 크기의 공간을 덮는 경우의 수는 $2 \times (n-1)$ 크기의 공간을 덮고 세로로 타일을 덮는 경우의 수(파란색 타일)과 $2 \times (n-2)$ 크기의 공간을 덮고 가로로 타일 2개를 덮는 경우(빨간색 타일)의 합이기 때문이다.
- ✓ $2 \times (n-2)$ 크기의 공간을 덮고 세로로 타일 2개를 덮는 경우를 생각하지 않는 이유는 $2 \times (n-1)$ 크기의 공간을 덮고 세로로 타일을 덮는 경우의 수와 중복되기 때문이다.



LIS (Longest Increasing Subsequence)

- ✓ 원소가 오름차순인 부분 배열 중 가장 길이가 긴 것(의 길이)을 구하는 문제이다.
- ✓ 예를 들어 배열 $[4, 1, 6, 8, 5]$의 LIS는 $[4, 6, 8]$이고 길이는 3이다.
- ✓ LIS의 길이를 구하기 위하여 $\mathrm{dp}[i]$를 배열의 i번째 원소로 끝나는 가장 긴 LIS의 길이로 정의하자.
- ✓ 우리가 구하는 정답은 배열의 길이가 n일 때 $\max(\mathrm{dp}[0], \mathrm{dp}[1], ...\mathrm{dp}[n-1])$이다.

LIS (Longest Increasing Subsequence)

- ✓ 배열 a의 LIS를 구해 보자.
- ✓ $\mathrm{dp}[0] = 1$로 초기값을 설정한다.
- ✓ 이후 $0 < j < n$인 j에 대하여 $\mathrm{dp}[j]$를 아래와 같이 계산해 준다.
  - $a[0]$부터 $a[j-1]$까지 원소 중 $a[j]$ 이하인 원소의 인덱스를 모두 구한다.
  - 구한 인덱스 중 $\mathrm{dp}[i]$의 최댓값을 x라 하면 $\mathrm{dp}[j] = x + 1$이 된다.
  - 만약 $a[j]$ 이하인 원소가 없다면 $\mathrm{dp}[j] = 1$이다.
- ✓ 마지막으로 $\max(\mathrm{dp}[0], \mathrm{dp}[1], \ldots \mathrm{dp}[n-1])$를 구하면 된다.
- ✓ 이 방법은 각 j에 대하여 배열을 전부 순회하므로 시간 복잡도가 $\mathcal{O}(n^2)$이다.
- ✓ 이분 탐색을 활용하여 시간 복잡도를 $\mathcal{O}(n \log n)$으로 줄일 수도 있다.



배낭 문제(Knapsack Problem)

- ✓ n개의 물건의 가치 c와 무게 w가 주어졌을 때 고른 총 물건의 무게가 W를 넘지 않도록 하면서 고른 물건의 총 가치의 합을 최대화하는 문제이다.
- ✓ 각 물건을 원하는 만큼 고를 수 있는 문제도 있고, 최대 1개까지만 고를 수 있는 문제도 있다.
- ✓ 최대 1개까지만 고를 수 있는 문제는 따로 0-1 배낭 문제라고도 부른다.



배낭 문제(Knapsack Problem)

- ✓ 0-1 배낭 문제를 어떻게 푸는지 알아보자.
- ✓ $\mathrm{dp}[i][j]$를 최대 무게가 j인 가방에서 첫 i번째 물건까지 고려했을 때 최대 가치로 설정하자.
- ✓ 이때 i는 1부터 시작하는 것으로 생각한다.
- ✓ dp 배열은 모두 0으로 초기화하면 된다.
- ✓ 각 물건들은 배낭에 넣는다, 혹은 배낭에 넣지 않는다의 두 가지 선택을 할 수 있다.



- 두 가지 선택 중 더 가치가 높은 쪽을 선택해야 하므로 점화식으로 나타내면

$$\mathrm{dp}[i][j] = \begin{cases} \max(\mathrm{dp}[i-1][j-w[i]] + c[i], \mathrm{dp}[i-1][j]) & (j \geq w[i]) \\ \mathrm{dp}[i-1][j] & (j < w[i]) \end{cases}$$

  이다.
- 이 점화식은 $1 \leq i \leq n$인 모든 i와 $0 \leq j \leq W$인 모든 j에 대해 순회하면서 값을 채우면 된다.
- 구하는 정답은 $\mathrm{dp}[n][W]$이다.



- 2차원 DP를 1차원 DP로 줄일 수도 있다.
- 위 점화식은 i번째 상태를 결정할 때 $i-1$번째 상태만 고려하므로 점화식에서 $[i]$를 없애도 된다.
- 그러면 점화식은 다음과 같이 변한다.

$$\mathrm{dp}[j] = \begin{cases} \max(\mathrm{dp}[j-w[i]]+c[i], \mathrm{dp}[j]) & (j \geq w[i]) \\ \mathrm{dp}[j] & (j < w[i]) \end{cases}$$

- 위 점화식과 동일하게 $1 \leq i \leq n$인 모든 i와 $0 \leq j \leq W$인 모든 j에 대해 순회하면서 값을 채우면 되지만, j는 W부터 0까지, 즉 역순으로 채워야 함에 유의하자.
- j를 증가하는 순서로 채운다면 넣은 물건을 또 넣을 수도 있다.
- 우리가 해결해야 하는 문제는 물건을 최대 1개씩만 골라야 함에 유의하자.



- 다음과 같이 코드를 짜면 된다.

```
for (int i = 1; i <= n; i++) {
  for (int j = 0; j <= W; j++) {
    if (j >= w[i]) dp[i][j] = max(dp[i-1][j-w[i]]+c[i], dp[i-1][j]);
    else dp[i][j] = dp[i-1][j];
  }
}
```

- 1차원 DP에서는 $j < w[i]$일 때 의미가 없으므로 j를 W부터 $w[i]$까지만 순회해도 충분하다.

```
for (int i = 1; i <= n; i++) {
  for (int j = W; j >= w[i]; j--) {
    dp[j] = max(dp[j-w[i]]+c[i], dp[j]);
  }
}
```

- **JUNGOL 2000** 동전교환
  앞에서 해설한 문제이다.
- **JUNGOL 3522** Tutorial 동적계획법(Dynamic Programming)
  피보나치 수 구하기
- **JUNGOL 1411** 두 줄로 타일 깔기
  $\mathrm{dp}[n]$을 가로 2칸, 세로 n칸인 크기의 판을 채우는 방법의 수로 정의하자.
- **JUNGOL 1520** 계단 오르기
  i번째 계단에 한 계단 밟아서 올라왔는지, 두 계단 밟아서 올라왔는지 구분하자.



- **JUNGOL 1848** 극장 좌석
  고정석이 없는 n개의 자리에 앉는 경우의 수는?
- **JUNGOL 1408** 전깃줄(초)
  전깃줄이 교차한다는 것이 어떤 의미인지 생각해 보자.
- **JUNGOL 1871** 줄 세우기
  움직이지 않아도 되는 아이는 최대 몇 명?
- **JUNGOL 12765** 연속한 1이 없는 이진수
  맨 오른쪽이 0일 때는 1을 붙일 수 있고 1이라면 1을 붙일 수 없다.



- **JUNGOL 1278** 배낭채우기2
  기본적인 0-1 배낭 문제.
- **JUNGOL 1077** 배낭채우기1
  넣은 물건을 또 넣어도 된다.
- **JUNGOL 2616** 앱(APP)
  배낭 문제의 응용.